Phase 5 Technical Architecture Blueprint: Autonomous Remediation Agent for Zero-Downtime Migrations

Executive Summary: The Strategic Imperative of Zero-Downtime Auto-Remediation

The maturation of enterprise DataOps practices has increasingly shifted focus from reactive incident management toward proactive, deterministic governance models. Over the preceding four phases of this architectural initiative, the deployment of a highly deterministic "DataOps CI/CD Gatekeeper" established a robust perimeter defense against destructive database operations. Operating natively within the GitHub Actions continuous integration environment, the Gatekeeper utilizes advanced Abstract Syntax Tree (AST) parsing via the sqlglot library to intercept developer Pull Requests (PRs).[1] By evaluating these PRs for disruptive SQL commands—most notably the DROP TABLE or ALTER TABLE DROP COLUMN directives—the system intercepts potentially catastrophic schema mutations before they reach production environments.

The true operational power of the Gatekeeper, however, lies in its topological awareness. By interfacing with an OpenMetadata (v1.12) instance via its REST API (/v1/lineage), the Gatekeeper traverses downstream lineage graphs up to three hops, specifically targeting assets annotated with critical governance tags such as Tier.Tier1.[2] Upon detecting a conflict where a dropped table directly jeopardizes a Tier 1 downstream asset (e.g., a critical dim_customer dimension table), the Gatekeeper enforces a hard block. It generates a detailed markdown artifact (report.md) outlining the blast radius and forcefully terminates the workflow using a custom exit 100 code.[4]

While highly effective at preventing data corruption, this purely defensive posture introduces substantial operational friction. When a PR is blocked, the developer is forced into a manual, high-latency remediation cycle. They must independently investigate the downstream schema requirements, reverse-engineer the dependencies, and manually craft compensatory SQL to preserve the pipeline's integrity. This manual intervention directly degrades developer velocity and extends the Mean Time To Resolution (MTTR) for pipeline changes.

Phase 5, termed the "Auto-Medic Pivot," addresses this operational friction by evolving the architecture from a static gatekeeper into an active, self-healing system. The core objective is the deployment of an Autonomous Remediation Agent (remediation_agent.py) designed to facilitate Zero-Downtime Schema Evolutions. This paradigm, heavily reliant on the "expand-contract" migration pattern, ensures that a database schema can evolve while continuous application and downstream data consumption remain uninterrupted.[6]

The Autonomous Remediation Agent bridges the gap between deterministic pipeline enforcement and Generative AI (GenAI). Triggered exclusively when the Gatekeeper blocks a PR, the agent ingests the blast radius report, queries OpenMetadata for precise downstream schema requirements, and utilizes a highly constrained Large Language Model (LLM) to generate a backward-compatible SQL CREATE VIEW statement.[8] This view acts as an ephemeral proxy, dynamically reconstructing the required columns and data types from the remaining table structures (or gracefully casting nulls) to ensure the downstream Tier 1 assets never experience a schema violation.[7]

The business value of this Phase 5 pivot is profound. It transforms a blocking CI/CD hurdle into a guided, auto-remediating workflow. Developers receive the exact SQL patch required to bypass the block, ensuring zero downtime for critical data consumers while maintaining the strict, immutable integrity guarantees of the existing Gatekeeper implementation.

Architectural Component Topology and Execution Flow

The integration of the Autonomous Remediation Agent into the existing CI/CD environment requires a highly decoupled, orthogonal component topology. A primary constraint of the Phase 5 initiative dictates that the core gatekeeper files (main.py, parser.py, api_client.py) are strictly locked and immutable. Consequently, the remediation agent cannot be integrated as a function call within the main application thread; it must operate as an independent, subsequent process within the overarching GitHub Actions workflow environment.

System Boundaries and Inter-Process Communication

Because the Gatekeeper and the Remediation Agent run as isolated Python processes, they cannot share in-memory state. Communication between the defensive and offensive layers of the architecture must occur asynchronously via the local file system and standard UNIX exit codes.

Architectural Component	Functional Classification	Primary Responsibility	Execution State
Gatekeeper Module	Deterministic Analyzer	Parses AST via sqlglot, traverses OpenMetadata lineage, halts CI/CD on violations.	Locked / Immutable

Artifact State File	Inter-Process Memory	The report.md file stores the localized blast radius, target table, and downstream FQNs.	Mutable (Append-Only)
GitHub Actions Runner	Workflow Orchestrator	Manages the execution state machine, catching exit 100 to trigger the remediation phase.	Configurable
Remediation Agent	Autonomous Generator	Extracts context from report.md, queries OpenMetadata, prompts the LLM, and appends SQL.	New Implementation
OpenMetadata (v1.12)	System of Record	Serves as the authoritative source for entity schema definitions and data types via REST API.	External Service
LLM Inference Engine	Reasoning Core	Evaluates schema constraints to generate standard-compliant CREATE VIEW SQL syntax.	Swappable (Local/Cloud)

Orchestration and the State Machine Lifecycle

The architectural lifecycle begins with the developer initiating a Pull Request. The GitHub Actions runner provisions the environment, checks out the repository, and executes the

Gatekeeper step. If the AST parser identifies a destructive operation that impacts a Tier 1 asset, the Gatekeeper generates the markdown report and executes a sys.exit(100).[4]

In a standard CI/CD configuration, an exit code of 100 (or any non-zero integer) immediately halts the workflow, skipping all subsequent steps and marking the job as failed.[4] To activate the Auto-Medic pivot, the runner's state machine is reconfigured using conditional execution semantics.[10] The workflow file defines the Remediation Agent step to execute strictly upon the failure of the Gatekeeper step.

Once triggered, the Remediation Agent assumes control of the runner environment. It initiates a precise file-system read operation to ingest report.md. Utilizing robust regular expressions, it parses the markdown table to extract the specific string identifiers of the dropped table and the Fully Qualified Name (FQN) of the impacted downstream asset.[12]

With these identifiers, the agent opens an authenticated network request to the OpenMetadata instance, targeting the specific endpoint required to extract the structural schema of the downstream asset.[2] This precise schema, formatted as a collection of column names and SQL data types, forms the core of the RAG (Retrieval-Augmented Generation) context.[14]

The agent then constructs a highly constrained inference prompt and dispatches it to the LLM. The architecture abstractly defines the LLM connection, standardizing on the OpenAI client specification.[15] This allows the agent to dynamically route the request either to a local Ollama service running the Gemma 4 model (for zero-cost continuous integration testing) or to the production Google Gemini API (for the final execution) based entirely on injected environment variables.[16]

Upon receiving the generated SQL payload from the LLM, the agent performs a secondary AST validation using sqlglot to ensure syntactic correctness.[1] If the payload passes, the agent executes an append operation to report.md, injecting the CREATE VIEW patch for the developer. Finally, the agent gracefully degrades by exiting with a 0 code, allowing the GitHub Actions runner to finalize the workflow as a failure (due to the original Gatekeeper block) while presenting the enriched, remediated report to the user.[18]

Trigger and Parse Mechanics: Bypassing the CI/CD Airgap

The deterministic handoff between the immutable Gatekeeper and the new Remediation Agent requires deep integration with the GitHub Actions workflow syntax and advanced text parsing techniques to ensure reliability.

Conditional Workflow Execution and Exit Code Trapping

The standard behavior of a shell command executing within a GitHub Actions run step is to enforce the -e flag, which causes the shell to exit immediately upon encountering a failure.[18]

The Gatekeeper's exit 100 is a deliberate signal. To utilize this signal without suppressing the failure state of the pipeline, the workflow leverages the if conditional attribute, specifically utilizing the steps.<step_id>.outcome syntax.[11]

The workflow YAML must be structured to assign an explicit id to the Gatekeeper step. The subsequent step, which invokes remediation_agent.py, uses the conditional: if: ${{ failure() && steps.gatekeeper.outcome == 'failure' }}.[21] This explicit condition ensures that the remediation script does not run if the workflow is cancelled manually (if: always()) or if a completely unrelated prior step (such as a linter) caused the failure.[10]

Furthermore, checking for the specific exit code can be achieved by utilizing bash traps or by configuring the Gatekeeper step to output its exit status to $GITHUB_OUTPUT prior to failing.[18] However, because the Gatekeeper code is locked, the runner must rely entirely on the outcome evaluation. This separation ensures that the CI/CD pipeline remains genuinely blocked—the remediation agent does not "fix" the pipeline automatically; it generates the code required for the human developer to implement the fix, maintaining the principle of human-in-the-loop governance.[22]

Advanced Artifact Parsing and Regex Extraction

Once the Remediation Agent is successfully triggered, its first challenge is reading the inter-process memory state: the report.md file. Markdown parsing in automated systems is notoriously brittle due to variations in whitespace, alignment padding, and newline characters across different operating systems (\n vs \r\n).[12]

The Gatekeeper generates a structured markdown table. Assuming a standard format where columns are delimited by pipe characters (|), the agent must extract specific fields. Utilizing Python's re module, the agent avoids heavyweight markdown parsing libraries to minimize dependencies, relying instead on highly optimized regular expressions.[23]

The target regex pattern must account for optional leading and trailing pipes, varied whitespace padding, and the specific row containing the dropped table data. A pattern such as ^\|?\s*([a-zA-Z0-9_]+)\s*\|\s*([a-zA-Z0-9_\.]+)\s*\| is employed, operating in a multiline context to iterate over the table rows.[12]

The agent isolates the first capture group (the Dropped Table name, e.g., raw_customer) and the second capture group (the Impacted Downstream FQN, e.g., postgres_prod.ecommerce.public.dim_customer).[23] Because FQNs in OpenMetadata are strictly formatted strings without spaces, the extraction logic includes rigorous validation assertions. If the regex engine returns a null match, or if the extracted FQN contains invalid characters, the agent triggers its graceful degradation protocol, preventing further execution based on corrupted state.[24]

Context Retrieval: Grounding via the OpenMetadata

API

The Generation phase of the Remediation Agent requires more than just the names of the tables; it requires a deep, structural understanding of the database. Large Language Models are highly susceptible to hallucinations—inventing column names, guessing data types, or assuming foreign key relationships that do not exist in reality.[14] To mitigate this, the architecture employs a strict Retrieval-Augmented Generation (RAG) pattern, using OpenMetadata as the factual anchor.[27]

Endpoint Targeting and Payload Specifications

OpenMetadata v1.12 provides multiple avenues for data extraction, including a Python SDK.[29] However, the Python SDK heavily relies on large Pydantic models generated from JSON schemas. To keep the Remediation Agent lightweight and decoupled from the specific version constraints of the openmetadata-ingestion package, the architecture utilizes standard Python requests to interface directly with the REST API.[30]

The agent targets the GET /v1/tables/name/{fqn} endpoint. This endpoint is specifically designed to bypass UUID lookups, allowing direct resolution of the table using the FQN extracted from the markdown report.[2]

Authentication is managed securely via JSON Web Tokens (JWT) provided by the OpenMetadata server. The GitHub Actions runner injects an environment variable containing the token, which the agent maps to the HTTP Authorization: Bearer <token> header.[30]

Schema Synthesization for Prompt Injection

The JSON payload returned by the /v1/tables/name/{fqn} endpoint is exhaustive, containing structure properties, location attributes, data quality parameters, and governance classifications.[24] Passing this entire raw JSON object into an LLM context window is inefficient, consumes excessive tokens, and risks distracting the attention mechanism of the model, leading to degraded reasoning.[31]

Therefore, the Remediation Agent executes a targeted extraction of the columns array. The parser iterates through this array, isolating only the name (the column identifier) and the dataType (the specific SQL type, such as VARCHAR, BIGINT, or TIMESTAMP).[29]

This extracted data is synthesized into a lightweight, highly readable Data Definition Language (DDL) representation. For example:

Column: customer_id | Type: BIGINT

Column: email_address | Type: VARCHAR

Column: last_login | Type: TIMESTAMP

By transforming the verbose JSON into a concise, deterministic schema map, the agent

creates a pristine factual ground truth. This schema representation defines the absolute contract that the LLM's generated CREATE VIEW statement must fulfill to ensure zero-downtime operation for the downstream asset.[7]

The Prompt Engineering Strategy: Forcing Deterministic Generation

The most critical—and volatile—component of the autonomous architecture is the Generative AI engine. Without rigorous prompt engineering, LLMs default to conversational paradigms, embedding their outputs in explanatory text, adding markdown code block delimiters (e.g., ```sql), or hallucinating unsupported SQL functions.[32] The Phase 5 architecture mandates a strict, deterministic output: exactly one syntactically valid CREATE VIEW statement, with zero conversational padding.

Architectural Constraints on the LLM

The objective is to implement the "expand-contract" database migration pattern. When a developer drops raw_customer, the downstream pipeline breaks because it is still querying that exact table name.[6] The LLM must generate a CREATE VIEW raw_customer AS SELECT... statement. This view acts as an interface proxy, fulfilling the downstream schema requirements by querying the remaining tables, or, if data is permanently lost, by using CAST(NULL AS <Type>) AS <column_name> to maintain the column contract and prevent downstream query failures.[7]

Multi-Layered Prompt Construction

The prompt architecture leverages several advanced techniques to suppress hallucinations and enforce formatting boundaries: Role-Based Prompting, Retrieval-Augmented Grounding (RAG), and Negative Constraint Fencing.[14]

1. The System Persona (Inference Bounding) The system prompt establishes the operational boundaries. It instructs the model to abandon its conversational nature and assume the persona of a deterministic SQL compilation engine.[26] *Directive:* You are an automated, deterministic database engine remediation module. Your sole function is to output raw, syntactically perfect SQL. You do not converse, explain, or format using Markdown. You operate in a high-stakes, zero-downtime CI/CD environment.

2. The RAG Context ("According To" Methodology) To prevent the model from guessing column names, the agent injects the synthesized OpenMetadata schema using the "According to..." prompting technique, which has been empirically shown to increase the Quoted Information Precision (QUIP) score and reduce fabrications.[35] *Context:* The developer has executed a DROP TABLE on '{dropped_table_name}'. According to the OpenMetadata repository, the downstream asset '{downstream_fqn}' absolutely requires the following schema to function correctly without downtime: \n {injected_schema_string}

3. Execution Directives and Step-Back Reasoning The model is given explicit logical steps to formulate the view, forcing a structured generation path.[35] *Instruction:* Task: Generate a backward-compatible SQL CREATE VIEW statement to replace the dropped table. Rules: 1. The view MUST be named exactly '{dropped_table_name}'. 2. The view MUST SELECT all necessary fields to satisfy the downstream schema provided above. 3. If specific source data is unavailable, you MUST synthesize the column using CAST(NULL AS <data_type>) AS <column_name> to ensure the structural contract is maintained and queries do not fail.

4. Strict Output Formatting and Negative Constraints The most persistent issue with code generation via LLMs is the inclusion of markdown backticks, which corrupt automated patching mechanisms. The prompt utilizes explicit negative constraints to prohibit this behavior.[32] *Formatting:* DELIVERY FORMAT RESTRICTION: You must deliver the response in plain, raw text. Do NOT use Markdown formatting. Do NOT wrap the SQL in triple backticks (```). Do NOT output the word 'sql'. The very first character of your response must be 'C' for 'CREATE VIEW'.

To further enforce this deterministic behavior, the agent configures the API payload with strict inference parameters. The temperature is set to 0.1 and top_p is set to 0.1. These settings drastically reduce the probability space of token selection, eliminating the model's "creativity" and forcing it to adhere strictly to the highest-probability, most logical syntax path.[33]

API Integration Specifications: The OpenAI Compatibility Layer

A core architectural requirement of Phase 5 is the establishment of a testing strategy that accounts for localized, cost-free execution while allowing a seamless transition to enterprise-grade production inference. Maintaining separate codebases or divergent API clients for local and cloud models introduces immense technical debt and testing inconsistencies. To solve this, the architecture capitalizes on the industry-wide adoption of the OpenAI API specification.[15]

Both Google's Gemini REST API and the local Ollama inference server expose endpoints that are fully compatible with the official openai Python SDK. By standardizing on this single library, the Remediation Agent can dynamically shift between inference engines by altering just three configuration variables: the base_url, the api_key, and the model identifier.[15]

Local Testing Environment: Ollama and Gemma 4

For local development and unit testing within the CI runner, the system relies on an Ollama daemon. The model of choice is Gemma 4 (e.g., gemma4:e2b or gemma4:e4b), an open-weights model optimized for coding and logical reasoning while maintaining a sufficiently small parameter footprint to run on standard CI/CD hardware.[16]

The agent detects the operational environment via a standard OS environment variable (e.g., ENV_MODE=local). Upon detecting local execution, the openai client is instantiated with the

Ollama routing configuration:

- **Base URL:** http://localhost:11434/v1 [39]
- **API Key:** ollama (The OpenAI SDK strictly requires an API key string, but the Ollama server ignores it, allowing dummy strings to pass authentication).[37]
- **Model:** gemma4 [16]

This configuration routes the highly constrained prompt directly to the local model. Developers can repeatedly test the extraction logic, the OpenMetadata RAG integration, and the prompt boundaries without incurring any cloud API costs or dealing with external network latency.

Production Execution: Google Gemini API

When the codebase is merged and executes in the production GitHub Actions environment, the workflow injects the production variables (ENV_MODE=production and the secure GEMINI_API_KEY stored within GitHub Secrets).

The agent dynamically reconfigures the OpenAI client instance to route requests to Google Cloud's infrastructure:

- **Base URL:** https://generativelanguage.googleapis.com/v1beta/openai/ [15]
- **API Key:** Loaded securely via os.getenv("GEMINI_API_KEY").[15]
- **Model:** gemini-2.5-flash [17]

The choice of gemini-2.5-flash is deliberate; it provides an optimal balance of low-latency generation and high-precision reasoning, ensuring the CI/CD pipeline does not stall waiting for inference.[42]

Crucially, because both backends adhere to the same schema, the Python code executing the inference—client.chat.completions.create(messages=[...])—remains completely untouched.[15] Furthermore, the payload structure for injecting system roles, user prompts, and temperature settings requires zero modification. This seamless interoperability guarantees that the behavior validated in local testing will flawlessly replicate in the production environment.

Once the LLM returns the completion payload, the agent extracts the string from response.choices.message.content. To counter any residual markdown hallucinations that may have bypassed the strict prompt engineering, the agent executes a final programmatic sanitization pass, stripping any leading or trailing ```sql strings before appending the text to the bottom of the report.md artifact.[32]

Failure Modes, System Resilience, and Graceful Degradation

The introduction of Agentic AI into a deterministic CI/CD pipeline introduces novel failure modes. Unlike traditional software that fails predictably on syntax errors, autonomous agents

can fail dynamically—they can experience reasoning loops, hallucination cascades, broken handoffs, and external dependency timeouts.[25] If the Remediation Agent crashes, hangs, or injects corrupted SQL into the report, it degrades the trust in the DataOps pipeline and exacerbates the problem it was designed to solve.

Therefore, the architectural blueprint mandates a rigorous framework for "Graceful Degradation." The absolute primary directive of the agent is: *Do no harm to the CI/CD pipeline's deterministic state*. The pipeline must remain blocked by the Gatekeeper's exit 100, regardless of what occurs during the remediation phase.[4]

Categorization of Agentic Failures and Architectural Mitigations

To achieve robust resilience, the system is designed to trap and mitigate specific classes of failure.

1. Context Retrieval Timeouts (Black-Box Blindness) If the OpenMetadata API experiences an outage, or if the network connection from the GitHub Actions runner times out, the agent suffers from "siloed context".[44] It cannot retrieve the downstream schema. Attempting generation without schema grounding guarantees a hallucination. *Mitigation:* All HTTP requests utilizing the requests library are wrapped with strict timeout parameters (e.g., timeout=10). If a requests.exceptions.Timeout is raised, the agent logs an execution warning, aborts the LLM prompt generation, and triggers the graceful exit protocol.[45]

2. Artifact Parsing Failures (Broken Handoffs) The interaction between the Gatekeeper and the Agent relies on the report.md file format. If a future update subtly alters the markdown table alignment, the regular expressions may fail to extract the FQN.[25] *Mitigation:* The Python extraction logic includes try...except blocks and null assertions. If regex.search() returns None, the agent recognizes the handoff is broken. It does not attempt to guess the table names; it immediately logs a parsing failure and executes the graceful exit protocol.

3. Hallucination Cascades and Syntactic Corruption Even with temperature=0.1 and aggressive prompt constraints, LLMs can theoretically generate invalid SQL—for instance, utilizing a proprietary function unsupported by the target database dialect, or failing to close a parenthesis.[28] Injecting broken SQL into the PR report introduces automation bias, tricking developers into deploying faulty code.[44] *Mitigation:* Before appending the LLM's payload to the file system, the Remediation Agent routes the string through a local installation of the sqlglot library. The agent calls sqlglot.parse_one(llm_output).[1] sqlglot acts as a deterministic verification layer.[33] If the LLM generated invalid syntax, parse_one() will raise a ParseError. The agent traps this exception, discards the corrupted LLM payload, logs a reasoning failure, and executes the graceful exit protocol.[27]

The Principle of Zero-Impact Failure via Exit Code Management

The cornerstone of the graceful degradation protocol is the management of the process exit code. In standard Python script design, a catastrophic failure results in a non-zero exit code

(e.g., sys.exit(1)). However, in this specific topological configuration, a non-zero exit code from the Remediation Agent would introduce a secondary, highly disruptive failure into the GitHub Actions log.[4]

Because the Remediation Agent only runs *after* the Gatekeeper has already failed the pipeline with exit 100, the workflow's overarching status is already correctly marked as a failure. If the agent encounters any of the aforementioned failure modes (Timeouts, Parsing Failures, Hallucinations), it is strictly programmed to utilize sys.exit(0).[4]

By trapping all internal exceptions and exiting cleanly with a 0, the agent degrades silently. It essentially "opts out" of the workflow without polluting the runner logs. The CI/CD pipeline remains securely blocked by the original Gatekeeper policy, and the developer is presented with the standard, un-remediated blast-radius report—exactly as the system functioned in Phases 1-4. This ensures that the introduction of GenAI into the enterprise pipeline strictly adheres to the principles of fail-safe security, maximizing the upside of automated remediation while mathematically capping the downside risk.[47]

Conclusion

The Phase 5 Technical Architecture Blueprint for the Autonomous Remediation Agent establishes a highly sophisticated, fault-tolerant bridge between deterministic DataOps governance and Generative AI. By orchestrating GitHub Actions runner states to conditionally bypass standard failure terminations, the system seamlessly hands off execution context from the locked, immutable Gatekeeper to the dynamic remediation module.

The integration with the OpenMetadata REST API serves as the critical anchor, guaranteeing that the LLM is factually grounded in the topological reality of the downstream schema, thereby neutralizing the primary risk of hallucinatory generation. Furthermore, the strategic implementation of an OpenAI-compatible interface ensures frictionless portability, allowing engineering teams to swap between local execution testing with Ollama and enterprise-grade production inference with the Google Gemini API without altering a single line of interaction logic.

Ultimately, through the application of rigorous, multi-layered prompt engineering constraints that enforce raw SQL output, and the deployment of comprehensive graceful degradation mechanisms that protect the CI/CD pipeline from agentic failure cascades, this architecture successfully transforms a rigid, defensive bottleneck into a proactive, zero-downtime enablement framework. The resulting system aggressively enforces data integrity standards while simultaneously augmenting developer velocity, resolving complex architectural dependencies autonomously at the speed of continuous deployment.

Works cited

1. sqlglot API documentation, accessed April 25, 2026, https://sqlglot.com/
2. Retrieve a Table - OpenMetadata Documentation, accessed April 25, 2026,

https://docs.open-metadata.org/v1.12.x/api-reference/data-assets/tables/retrieve
3. Tables - OpenMetadata Documentation, accessed April 25, 2026, https://docs.open-metadata.org/v1.12.x/api-reference/data-assets/tables
4. Setting exit codes for actions - GitHub Docs, accessed April 25, 2026, https://docs.github.com/actions/creating-actions/setting-exit-codes-for-actions
5. Incomplete output - ##[error]Bash exited with code '100'. · Issue #8729, accessed April 25, 2026, https://github.com/actions/runner-images/issues/8729
6. Plan a Zero-Downtime Database Migration - Prompt Guide, accessed April 25, 2026, https://prompt-guide.com/en/prompts/migrer-base-donnees-zero-downtime
7. Zero-Downtime Schema Evolution in SQL Databases - Medium, accessed April 25, 2026, https://medium.com/@CodeWithHannan/zero-downtime-schema-evolution-in-sql-databases-behind-the-scenes-of-production-migrations-7e921ff0c0c2
8. Creating views on existing tables with the SQL editor - IBM, accessed April 25, 2026, https://www.ibm.com/docs/en/idr/11.4.0?topic=tables-creating-views-existing-sql-editor
9. CREATE VIEW (Transact-SQL) - SQL Server - Microsoft Learn, accessed April 25, 2026, https://learn.microsoft.com/en-us/sql/t-sql/statements/create-view-transact-sql?view=sql-server-ver17
10. How to run a github-actions step, even if the previous step fails, accessed April 25, 2026, https://stackoverflow.com/questions/58858429/how-to-run-a-github-actions-step-even-if-the-previous-step-fails-while-still-f
11. GitHub Actions: Dealing with a failing step in a workflow | Antti Viitala, accessed April 25, 2026, https://aviitala.com/posts/github-actions-failing-step-workflow/
12. Regex for Markdown Table Syntax? - github - Stack Overflow, accessed April 25, 2026, https://stackoverflow.com/questions/9837935/regex-for-markdown-table-syntax
13. Extract markdown table from a markdown string - Regex101, accessed April 25, 2026, https://regex101.com/r/8pNnaG/1/codegen?language=csharp
14. accessed April 25, 2026, https://airbyte.com/agentic-data/prevent-llm-hallucinations
15. OpenAI compatibility | Gemini API - Google AI for Developers, accessed April 25, 2026, https://ai.google.dev/gemini-api/docs/openai
16. Run Gemma with Ollama | Google AI for Developers, accessed April 25, 2026, https://ai.google.dev/gemma/docs/integrations/ollama
17. Import OpenAI-Compatible Google Gemini API - Microsoft Learn, accessed April 25, 2026, https://learn.microsoft.com/en-us/azure/api-management/openai-compatible-google-gemini-api
18. How can I capture exit code of run command and reuse in another job, accessed April 25, 2026, https://github.com/orgs/community/discussions/46992

19. Forcing a failure in GitHub Actions based on a condition - Tim Heuer, accessed April 25, 2026, https://timheuer.com/blog/manually-force-a-failure-in-github-action-step/
20. How to Retry Failed Steps in GitHub Action Workflows - This Dot Labs, accessed April 25, 2026, https://www.thisdot.co/blog/how-to-retry-failed-steps-in-github-action-workflows
21. How to Handle Step and Job Errors in GitHub Actions - Ken Muse, accessed April 25, 2026, https://www.kenmuse.com/blog/how-to-handle-step-and-job-errors-in-github-actions/
22. generative ai for pipeline repair: automating corrective actions in self, accessed April 25, 2026, https://www.researchgate.net/publication/396161486_GENERATIVE_AI_FOR_PIPELINE_REPAIR_AUTOMATING_CORRECTIVE_ACTIONS_IN_SELF-_HEALING_DATA_SYSTEMS
23. Extract data from markdown table - App Building - Retool Forum, accessed April 25, 2026, https://community.retool.com/t/extract-data-from-markdown-table/28463
24. Table - OpenMetadata Standards, accessed April 25, 2026, https://openmetadatastandards.org/data-assets/databases/table/
25. 7 AI Agent Failure Modes and How to Prevent Them | Galileo, accessed April 25, 2026, https://galileo.ai/blog/agent-failure-modes-guide
26. How prompt engineering reduces hallucinations in AI models, accessed April 25, 2026, https://media-beats.com/en/prompt-engineering-hallucinations-ai-models/
27. 5 Practical Techniques to Detect and Mitigate LLM Hallucinations, accessed April 25, 2026, https://machinelearningmastery.com/5-practical-techniques-to-detect-and-mitigate-llm-hallucinations-beyond-prompt-engineering/
28. How to Reduce LLM Hallucinations in Enterprise Applications, accessed April 25, 2026, https://www.ksolves.com/blog/artificial-intelligence/reduce-llm-hallucinations
29. Python SDK - OpenMetadata Documentation, accessed April 25, 2026, https://docs.open-metadata.org/v1.12.x/api-reference/sdk/python
30. OpenMetadata Python API Docs | dltHub, accessed April 25, 2026, https://dlthub.com/context/source/openmetadata
31. Prompt Engineering for a Better SQL Code Generation With LLMs, accessed April 25, 2026, https://medium.com/datamindedbe/prompt-engineering-for-a-better-sql-code-generation-with-llms-263562c0c35d
32. How to prevent GPT from outputting responses in Markdown format?, accessed April 25, 2026, https://community.openai.com/t/how-to-prevent-gpt-from-outputting-responses-in-markdown-format/961314
33. Eliminating LLM Hallucinations: A Multi-Layer Defense Strategy That, accessed

April 25, 2026, https://medium.com/@murali.nandigama/eliminating-llm-hallucinations-a-multi-layer-defense-strategy-that-actually-works-1702febb9e4d
34. The importance of prompt engineering in preventing AI hallucinations, accessed April 25, 2026, https://alfapeople.com/us/importance-of-prompt-engineering-preventing-ai-hallucinations/
35. Reducing AI Hallucinations: 6 Prompt Engineering Techniques That, accessed April 25, 2026, https://medium.com/@aysan.nazarmohamady/reducing-ai-hallucinations-6-prompt-engineering-techniques-that-actually-work-16b583797bd0
36. Improve Accuracy and Reduce Hallucinations with a ... - PromptHub, accessed April 25, 2026, https://www.prompthub.us/blog/improve-accuracy-and-reduce-hallucinations-with-a-simple-prompting-technique
37. Cloud - Ollama's documentation, accessed April 25, 2026, https://docs.ollama.com/cloud
38. Converting Gemini API calls to Ollama. - Reddit, accessed April 25, 2026, https://www.reddit.com/r/ollama/comments/1g14gih/converting_gemini_api_calls_to_ollama/
39. Integrating Ollama with Python: REST API and Python Client Examples, accessed April 25, 2026, https://www.glukhov.org/llm-hosting/ollama/ollama-python-examples/
40. Using Gemini CLI with a Local LLM - DEV Community, accessed April 25, 2026, https://dev.to/polar3130/using-gemini-cli-with-a-local-llm-5f5l
41. Gemini is now accessible from the OpenAI Library, accessed April 25, 2026, https://developers.googleblog.com/en/gemini-is-now-accessible-from-the-openai-library/
42. Getting started with the Gemini API OpenAI compatibility - Colab, accessed April 25, 2026, https://colab.sandbox.google.com/github/google-gemini/cookbook/blob/main/quickstarts/Get_started_OpenAI_Compatibility.ipynb
43. Expose Gemini CLI endpoints as OpenAI API with Cloudflare Workers, accessed April 25, 2026, https://github.com/GewoonJaap/gemini-cli-openai
44. 12 Failure Patterns of Agentic AI Systems - Concentrix, accessed April 25, 2026, https://www.concentrix.com/insights/blog/12-failure-patterns-of-agentic-ai-systems/
45. Agentic AI for DataOps: From Alert Fatigue to Fully Automated, accessed April 25, 2026, https://www.acceldata.io/blog/agentic-ai-for-dataops-from-alert-fatigue-to-fully-automated-incident-remediation
46. sqlglot/posts/onboarding.md at main - GitHub, accessed April 25, 2026, https://github.com/tobymao/sqlglot/blob/main/posts/onboarding.md
47. Can AI Autonomously Build, Operate, and Use the Entire Data Stack?, accessed April 25, 2026, https://arxiv.org/html/2512.07926v1